Low-Code → Pro-Code: End-to-End Workflow & Change Management

This document describes the **complete lifecycle** of an agent-based solution built with the Low-Code-to-Pro-Code pattern — from a business user authoring an agent in **Microsoft Foundry Agent Service**, through **YAML hand-off**, **orchestration design**, **code generation**, **evaluation**, and **CI/CD to production** — and, critically, how a change made by a low-code author **propagates** all the way to the production application.

> TL;DR: Foundry Agent Service is the **source of truth for individual agent definitions**. The pro-code repository is the **source of truth for orchestration, tool logic, evaluation, and deployment**. A versioned YAML file is the **contract** between the two worlds. Every change flows through Git, is regenerated deterministically, is re-evaluated, and is promoted through environments by CI/CD.

---

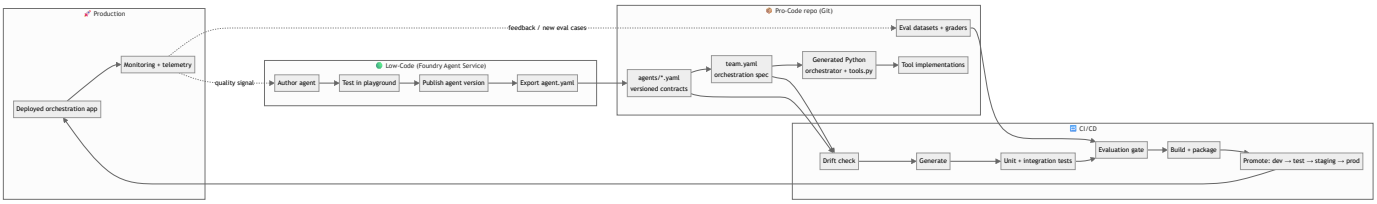
1. Personas & responsibilities

Persona	Owns	Works in
Low-code author (business SME / citizen developer)	Agent instructions, model choice, tool contracts, knowledge sources	Foundry Agent Service (portal)
Pro-code / platform engineer	Orchestration pattern, tool implementations, wiring, non-functionals	This repo (`yaml_to_sdk` , generated projects)
ML / evaluation engineer	Eval datasets, graders, quality gates	Evaluation pipeline
DevOps / SRE	CI/CD, environments, promotion, rollback, monitoring	Pipelines + Azure
Product / change owner	Approvals, changelog, release sign-off	Git PRs / work items

A single change often touches several personas — the workflow below is designed so that each hand-off is an explicit, reviewable, versioned artifact rather than a manual copy-paste.

---

2. The big picture



The rest of this document walks each stage in detail.

---

3. Stage 1 — Author the agent (Low-Code, Foundry Agent Service)

The low-code author works entirely in the Foundry Agent Service portal:

1. **Create the agent** — name, description, model deployment (e.g. `gpt-5`), and system instructions.
2. **Define tool contracts** — function tools (name, description, JSON-Schema parameters, `strict` mode) and/or MCP / knowledge-base tools (server URL + project connection).
3. **Configure reasoning** — e.g. `reasoning.effort: low|medium|high`.
4. **Test in the playground** — iterate on prompt and tool contracts until behaviour is right.
5. **Publish a version** — Foundry assigns a monotonically increasing version.

The author is responsible for **the contract**, not the implementation. Function tools are declared as schemas; the actual business logic (calling an ITSM system, network telemetry API, etc.) is implemented later by pro-code engineers.

---

4. Stage 2 — Export & share the YAML (the contract)

Each published agent version can be exported as a declarative YAML. This is the **hand-off artifact**. Its header carries the versioning anchors:

```yaml
object: agent.version
id: vf-triage-tool-agent:2      # <name>:<version>
name: vf-triage-tool-agent
version: "2"                     # Foundry version integer
description: "Network fault triage and remediation agent for Vodafone"
created_at: 1779097149

definition:
  kind: prompt
  model: gpt-5
  instructions: | ...
  reasoning:
    effort: low
  tools:
    - type: function
      name: get_incident
      ...
    - type: mcp
      server_label: kb_regulationpolciies_ziw96
      server_url: https:// ... /mcp?api-version=2025-11-01-Preview
      project_connection_id: kb-regulationpolciies-ziw96
```

Key fields for change management:

Field	Purpose
`name`	Stable identity of the agent across versions
`version`	Foundry-assigned version integer — increments on every publish
`id` (`name:version`)	Fully-qualified, immutable reference to one specific version
`definition.*`	The behavioural contract (model, instructions, tools)

How it is shared: the exported YAML is committed to the pro-code repository under a well-known folder, e.g.:

```
repo/
  agents/
    vf-triage-tool-agent.yaml     # tracks the latest agreed version
    vf-comms-agent.yaml
```

Sharing via **Git (a pull request)** — not email or chat — is what makes the hand-off auditable and reversible. The PR that lands a new agent YAML is the point where change management begins (see §10).

---

5. Stage 3 — Decide the orchestration pattern

Multiple agents rarely act alone. The pro-code team together with the low-code team choose **how** the agents cooperate, using the Azure AI agent orchestration patterns as the decision framework. This solution supports four Microsoft Agent Framework orchestrations:

Pattern	Coordination	Use when
sequential *(default)*	Pipeline; each agent consumes the previous output	Clear linear dependencies, progressive refinement
concurrent	All agents run in parallel; results aggregated	Independent perspectives; latency-sensitive
group_chat	Shared thread with a chat manager	Consensus-building, maker-checker validation
handoff	Control transfers dynamically between agents	The right specialist emerges during processing

Default pattern

Choosing a pattern is a design decision, but it must never be a blocker. The pro-code team implements `sequential` **as the default orchestration in the codebase**: if a `team.yaml` omits the `pattern` field — or agents are handed over before a pattern has been agreed — the generator falls back to `sequential`. This guarantees that any valid set of agent contracts produces a working, review-ready orchestration out of the box, and the team can switch patterns later when the coordination needs are clear.

- **Why sequential?** It is the simplest, most predictable pattern (a linear pipeline), the easiest to test and reason about, and a safe baseline before moving to `concurrent`, `group_chat`, or `handoff`.
- **How it's implemented:** the `pattern` field defaults to `sequential` in the generator schema (`OrchestrationDefinition`), so `pattern` is optional in `team.yaml` and the CLI `--pattern` flag is optional. Explicitly setting a pattern always overrides the default.
- **Changing it later** is a one-line edit to `team.yaml` that flows through the normal Git + CI/CD process and re-runs evaluation like any other change.

The decision is captured as a **team specification** — itself a versioned YAML — that references the individual agent contracts:

```yaml
# team.yaml
name: vf-triage-team
description: "Triage the fault, then notify customers."
orchestration:
  # `pattern` is OPTIONAL — defaults to 'sequential' if omitted.
  # Override with: concurrent | group_chat | handoff
  pattern: sequential
  task: "Handle incident INC-4291 end to end."
  agents:
    - ./agents/vf-triage-tool-agent.yaml
    - ./agents/vf-comms-agent.yaml
  max_rounds: 6          # group_chat
  # start_agent / handoffs  # handoff
```

The `team.yaml` is the **orchestration source of truth**. Changing the pattern, adding an agent, or re-ordering a pipeline is a change to this file and flows through the same Git + CI/CD process as an agent change.

---

6. Stage 4 — Convert YAML → Python (code generation)

The pro-code engineer runs the generator to turn the declarative contracts into a runnable Agent Framework project:

```bash
# Team (multi-agent) generation
python -m yaml_to_sdk --team-yaml team.yaml --output-dir generated_agents

# Or straight from agent files + a pattern
python -m yaml_to_sdk \
    --agents agents/vf-triage-tool-agent.yaml agents/vf-comms-agent.yaml \
    --pattern sequential --name vf-triage-team \
    --output-dir generated_agents
```

Generated layout:

```
vf_triage_team/
  team.yaml                 # pattern + which agents (loaded at runtime)
  agents/
    vf_triage_tool_agent.yaml    # the declarative agent contracts, copied in
    vf_comms_agent.yaml
  src/
    orchestrator.py         # ONE runtime loader: reads the YAMLs, builds every
                            # agent in-process, wires the chosen pattern
    tools.py                # ONE file: all tool stubs + TOOL_REGISTRY
    config.py               # env-driven configuration
  tests/
    test_team.py
  pyproject.toml
  requirements.txt
  .env.example
  Dockerfile
  README.md
```

What the generator produces:

- **`orchestrator.py`** — a single, *pattern-agnostic* file. At run time it loads `team.yaml`, traverses each `agents/*.yaml`, and builds one `Agent` per YAML in-process: instructions and model come from the YAML; function tools are resolved by name from `TOOL_REGISTRY`; MCP / knowledge tools are constructed from the YAML as `MCPStreamableHTTPTool`. It then selects the right Agent Framework builder (`SequentialBuilder`, `ConcurrentBuilder`, `GroupChatBuilder`, or `HandoffBuilder`) based on the `pattern` in `team.yaml`.
- **`tools.py`** — every function tool declared across the agents, de-duplicated into one place as typed `@tool` stubs with `TODO` bodies, plus a `TOOL_REGISTRY: {tool_name: impl}` map.
- **`team.yaml`** + **`agents/*.yaml`** — the contracts themselves, copied into the project so the orchestrator can load them at run time.

> **No per-agent Python files.** Agents are defined by their YAML and built dynamically by the single `orchestrator.py`. Adding, editing, or re-ordering an agent is a **YAML change**; changing the pattern is a one-line edit to `team.yaml`. `orchestrator.py` is a deterministic, regenerable artifact — put business logic in `tools.py` (next stage) where it is safe.

---

7. Stage 5 — Implement tools & stitch the solution

`tools.py` is the single place to add behaviour. It contains one typed `@tool` stub per unique function tool, wired to agents by name through `TOOL_REGISTRY`:

```
vf_triage_team/
  agents/*.yaml              # CONTRACTS — edit to change an agent (instructions, model, tools)
  team.yaml                  # PATTERN + membership
  src/
    orchestrator.py          # GENERATED — the runtime loader/builder (regenerable)
    tools.py                 # HAND-WRITTEN — implement tool bodies here
```

Implement each stub directly (or delegate to your own service modules):

```python
# src/tools.py  (generated stub — fill in the body)
from agent_framework import tool

@tool
async def get_incident(incident_id: Annotated[str, "Incident ID, e.g. INC-4291"]) -> dict:
    # TODO → call your ITSM system
    return await itsm.get_incident(incident_id)   # ← your logic lives here

# The orchestrator looks each agent's function tools up here at build time:
TOOL_REGISTRY = {"get_incident": get_incident, ...}
```

Because `orchestrator.py` is generated and `tools.py` is hand-written, **regeneration is safe and boring**: re-running the generator rewrites the loader but leaves your implementations untouched. MCP / knowledge tools need no code — they are attached automatically from the YAML.

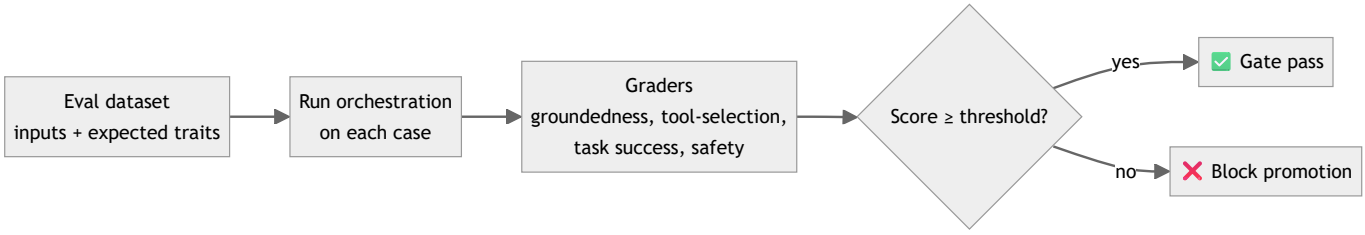
The engineer then:

1. Implements each tool body in `tools.py` (or delegates to a service module).
2. Adds non-functionals: auth, retries/timeouts, structured logging, guardrails.
3. Adjusts agents by editing `agents/*.yaml`, and the pattern/routing in `team.yaml`.
4. Runs the app locally (`python -m src.orchestrator`) against dev resources.

---

8. Stage 6 — Evaluation pipeline

Because agent outputs are **non-deterministic**, quality is verified with scored evaluations rather than exact-match assertions.



Artifacts that live in the repo and are versioned alongside the agents:

- `eval/datasets/*.jsonl` — representative tasks per agent **and** per team flow.
- `eval/graders/*` — rubric / LLM-as-judge / custom scorers.
- `eval/thresholds.yaml` — minimum passing scores per metric (the **quality gate**).

The evaluation runs at two levels:

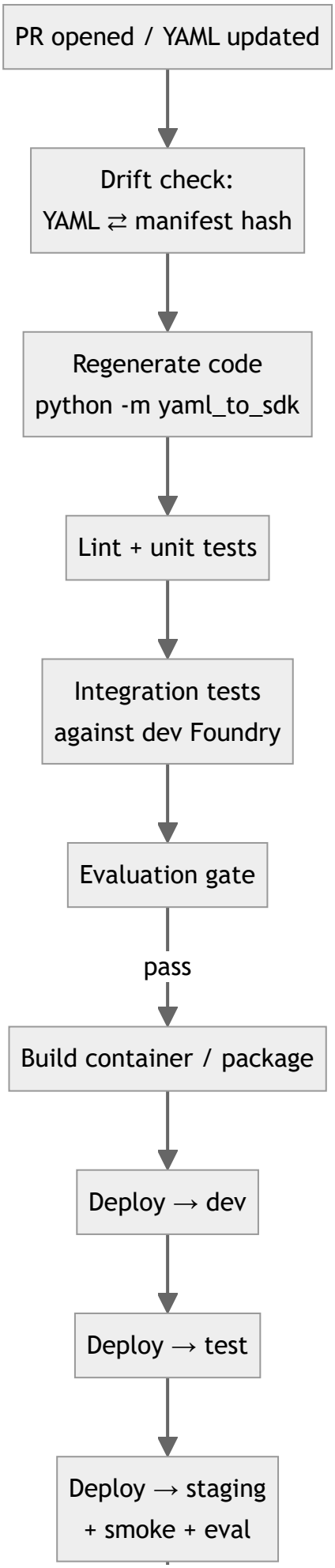
1. **Per-agent** — does each agent still satisfy its contract?
2. **End-to-end (team)** — does the orchestration produce the right business outcome?

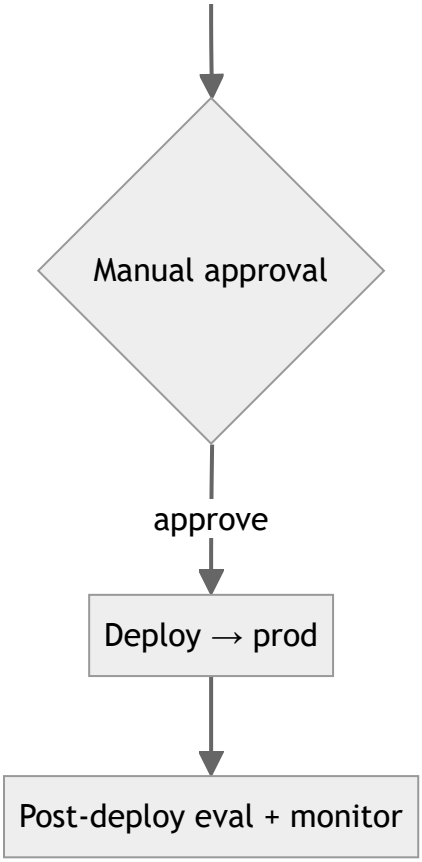
The eval results are stored as a **build artifact** and attached to the PR / release, giving a quality record for every version.

---

9. Stage 7 — CI/CD path to production

Every push and PR runs the pipeline. Promotion is gated on tests **and** evaluation.


PR opened / YAML updated
Drift check:
YAML ⇄ manifest hash
Regenerate code
python -m yaml_to_sdk
Lint + unit tests
Integration tests
against dev Foundry
Evaluation gate
pass
Build container / package
Deploy → dev
Deploy → test
Deploy → staging
+ smoke + eval



Pipeline stages in words:

1. **Drift check** — fail the build if the committed generated code doesn't match what the current YAML would produce (see §10.3). This guarantees the Python always reflects the contract.
2. **Regenerate** — run the generator so the build uses a fresh `orchestrator.py`; hand-written `tools.py` is untouched.
3. **Tests** — unit tests for tool logic; integration tests for the orchestration.
4. **Evaluation gate** — run the eval suite; block if any metric is below threshold.
5. **Build & package** — container image (the generated `Dockerfile`) or module artifact.
6. **Promote** — deploy to **dev** → **test** → **staging** → **prod**, with automated smoke tests and (at least at staging) a full eval run. Production requires a **manual approval**.
7. **Post-deploy** — run a canary eval and wire monitoring/telemetry.

Environments each have their own Foundry endpoint/model deployment and configuration via environment variables (`FOUNDRY_PROJECT_ENDPOINT`, `FOUNDRY_MODEL`, …).

---

10. Change management & versioning

This is the heart of the workflow: **what happens when a low-code author changes an agent?**

10.1 Versioning model

Three independent version streams, all reconciled in Git:

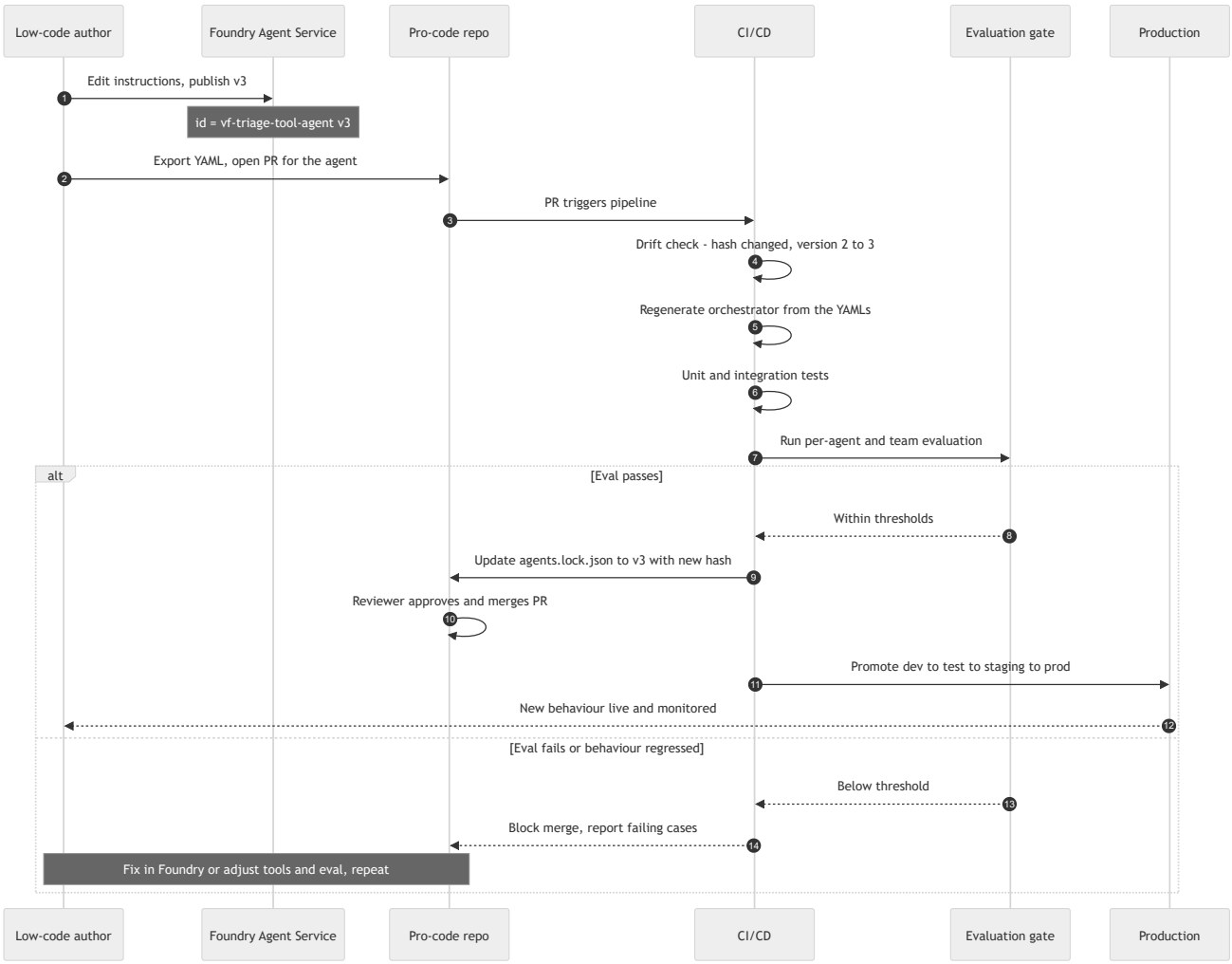
Level	Version source	Example
Agent	Foundry `version` integer (in the YAML)	`vf-triage-tool-agent:3`
Team / orchestration	Semantic version of `team.yaml`	`vf-triage-team v1.4.0`
Release	Git tag on the deployed solution	`v2026.07.0`

A **manifest / lockfile** ties them together and records the content hash of each YAML so the system can detect *any* change, not just a version bump:

```
// agents.lock.json  (committed, machine-generated)
{
  "team": { "name": "vf-triage-team", "version": "1.4.0" },
  "agents": [
    { "name": "vf-triage-tool-agent", "version": "2", "sha256": "9f3c…", "source": "agents/vf-
    { "name": "vf-comms-agent",       "version": "1", "sha256": "1a7e…", "source": "agents/vf-
  ],
  "pattern": "sequential",
  "generated_at": "2026-07-21T10:00:00Z"
}
```

10.2 The propagation flow (a low-code change reaching production)

Suppose the low-code author edits the triage agent's instructions and republishes it as **version 3**.



Step-by-step:

1. **Author publishes v3** in Foundry. The `version` becomes `3` and `id` becomes `vf-triage-tool-agent:3`. Nothing in production changes yet — Foundry is the contract source, not the runtime for the orchestration.
2. **Export & PR.** The updated YAML is committed to `agents/` via a pull request. The PR is the **change record** (who, what, why, when).
3. **CI detects the change.** The drift check compares the YAML's content hash against `agents.lock.json`. A changed hash (and/or a bumped `version`) marks the agent as *modified* and forces regeneration + full evaluation — you cannot merge a changed contract without re-qualifying it.
4. **Regenerate.** The generated `orchestrator.py` is rewritten from the YAMLs; hand-written `tools.py` stays intact, so no business logic is lost. If the change **added or renamed a tool**, the regenerated `tools.py` gains a new stub (or the orchestrator references a tool name with no implementation) → tests fail loudly, signalling the engineer to implement it. This is the safety net that prevents silent breakage.
5. **Re-run evaluation.** Both the per-agent and the end-to-end team evals run again against the new behaviour. Instruction changes frequently shift outputs, so this gate is mandatory.

6. **Promote or block.** If evals pass, `agents.lock.json` is updated and the PR merges; CI promotes through environments to production behind a manual approval. If evals fail, the PR is blocked with the failing cases attached, and the change loops back to the author or the pro-code team.

10.3 Drift detection in CI

Drift = the committed generated code (or lockfile) no longer matches what the current YAML would produce. The CI step regenerates into a temp dir and diffs:

```
python -m yaml_to_sdk --team-yaml team.yaml --output-dir .ci_generated --force
# Compare the generated orchestrator against the committed one; ignore hand-written tools.py.
git diff --no-index --exit-code src/orchestrator.py .ci_generated/*/src/orchestrator.py
# Recompute agent hashes and compare to agents.lock.json
python scripts/check_lock.py agents/ agents.lock.json
```

If drift is found, the build fails with a clear message: *"YAML changed but generated code / lockfile is stale — run the generator and commit."* This makes "the code always reflects the contract" a mechanically enforced invariant rather than a hope.

10.4 Which changes require what

Change made by low-code author	Regenerate?	Re-implement tools?	Re-evaluate?	Typical version bump
Instruction / prompt wording	Yes (stub prompt)	No	**Yes** (behaviour shift)	Agent minor
Model or reasoning effort	Yes	No	**Yes**	Agent minor
Add / rename a function tool	Yes	**Yes** (new impl)	**Yes**	Agent major
Change a tool's parameters	Yes	Likely	**Yes**	Agent major
Add / change an MCP source	Yes	No (config)	**Yes**	Agent minor
Orchestration pattern change (`team.yaml`)	Yes (orchestrator)	No	**Yes** (flow-level)	Team major

10.5 Rollback

Because everything is versioned in Git and images are immutable:

- **Fast rollback:** redeploy the previous release tag / container image — production returns to the last known-good version in minutes.
- **Contract rollback:** revert the PR that introduced the new agent YAML; CI regenerates and re-evaluates the prior version, restoring `agents.lock.json` .

- Foundry retains historical versions (`name:1` , `name:2` , …), so the exact prior contract is always recoverable.

10.6 Governance & audit

- Every change is a **pull request** with a reviewer and a linked work item.
- The **evaluation report** is attached to each PR/release as the quality evidence.
- `agents.lock.json` + Git history provide a complete, timestamped audit trail of *which agent version, at which content hash, was deployed in which release.*
- Optional: require the low-code author to include a short **change note** (what changed and why) in the PR description so downstream teams understand intent.

---

11. Roles across the lifecycle (RACI summary)

Activity	Low-code author	Pro-code eng	Eval eng	DevOps
Author / publish agent	**R/A**	C	I	I
Export & PR the YAML	**R**	C	I	I
Choose orchestration pattern	C	**R/A**	C	I
Generate + implement tools	I	**R/A**	I	I
Build eval datasets & gates	C	C	**R/A**	I
CI/CD & promotion	I	C	C	**R/A**
Approve production release	C	C	C	**A**
Rollback	I	C	I	**R/A**

(R = Responsible, A = Accountable, C = Consulted, I = Informed.)

12. Summary

1. **Foundry Agent Service** is the source of truth for **individual agent contracts**; each published version is exported as a **versioned YAML**.
2. The **pro-code repo** owns **orchestration, tool logic, evaluation, and deployment**, with `team.yaml` as the orchestration contract.
3. The **generator** deterministically turns YAML into Agent Framework Python; generated code is disposable, hand-written `tools.py` is durable.

4. **Evaluation gates** qualify every change; **CI/CD** promotes through environments to prod.
5. **Change management** is enforced by Git, a **hash-based lockfile**, **drift detection**, and **mandatory re-evaluation** — so a low-code author's edit propagates safely and traceably from Foundry all the way to the production app, and can be rolled back at any time.

References

- AI agent orchestration patterns (Azure Architecture Center)
- Microsoft Agent Framework workflow orchestrations